PREDICTIVE MODELLING · PROJECT REPORT

F1 Race Strategy Simulator

Predicting lap times from real race data and choosing the pit strategy

The question this report answers:

THE PROBLEM IN ONE SENTENCE

A few laps into a Formula 1 race, can a model predict the rest of the race well enough to choose when to change tyres and which tyres to fit?

$$\hat{s} \;=\; \arg\min_{s \,\in\, \mathcal{S}} \hat{R}(s), \qquad \hat{R}(s) \;=\; \sum_{l=l_0}^{L} \hat{t}_l(s) \;+\; Pk(s)$$

In words: among all allowed strategies, choose the one with the smallest predicted race time. Race time is the sum of the predicted lap times plus the time lost in the pit lane.

55,215	0.835 s	71%	0.57%
real racing laps used, from four seasons	lap-time error (RMSE) on the unseen 2025 season	of the error of assuming constant pace removed	error on total race time

Data: official Formula 1 timing for 2022 to 2025, read with the FastF1 library.

Contents

1 Objective 4

1.1 Background: what a pit strategy is 4

1.2 Problem statement 4

1.3 The problem written as mathematics 4

1.4 What kind of problem this is 5

1.5 Objectives 5

2 Data description 6

2.1 Source 6

2.2 Cleaning: from raw laps to usable laps 6

2.3 The baseline and the target 7

2.4 Inputs to the model 8

2.5 Training, validation and test sets 8

2.6 Pit-stop time loss 9

2.7 A first look at the target 9

3 Methods and mathematics 11

3.1 Notation 11

3.2 Supervised regression 11

3.3 Measuring error 12

3.4 Reference models 12

3.5 Linear regression 13

3.6 Regression trees 14

3.7 Gradient boosting 15

3.8 Overfitting and model selection 17

3.9 The coefficient of determination, R^2 18

3.10 From one lap to a whole race 19

3.11 Optimisation: searching the strategies 19

3.12 The structure of the error 20

3.13 Uncertainty of a race time 21

3.14 Deciding between two strategies 22

4 Results 24

4.1 Model comparison on the validation season . . . 24
4.2 How complex should the trees be? . . . 24
4.3 Final test on the 2025 season . . . 25
4.4 What the model learned about tyres . . . 26
4.5 Structure of the error . . . 27
4.6 Race-time prediction . . . 27
4.7 Recommended strategy against the teams' choices . . . 28
4.8 A worked race: 2025 Saudi Arabian Grand Prix . . . 29

5 Conclusion . . . 31
5.1 What was found . . . 31
5.2 Limitations . . . 31
5.3 Possible extensions . . . 32
5.4 Reproducing the results . . . 32

SECTION 1

Objective

This section explains the problem. It starts with the small amount of Formula 1 background needed to follow the rest of the report, and then writes the problem as mathematics.

1.1 Background: what a pit strategy is

A Formula 1 race is a fixed number of laps of a circuit, usually between 50 and 70, and lasts about an hour and a half. The driver who covers those laps in the least total time wins. Four facts decide how the lap time changes during a race:

- **Tyres wear out.** A new set of tyres has the most grip. Every lap removes some rubber, so the car gets slower as the tyres age. This is called tyre degradation.
- **There are three kinds of dry tyre.** They are called soft, medium and hard. Soft tyres are the fastest but wear out quickly. Hard tyres are slower but last longer. Medium is in between.
- **Fuel burns.** Cars start with all the fuel for the race and cannot refuel. A car is therefore heaviest at the start and gets lighter, and faster, on every lap.
- **A pit stop costs time.** To fit new tyres the driver must go through the pit lane at a limited speed and stop. Compared with staying on the track this costs about 20 to 25 seconds.

There is also a rule: in a dry race every driver must use at least two different kinds of dry tyre. So every driver must stop at least once.

A **pit strategy** is the plan that answers two questions: on which lap do we stop, and which tyres do we fit? Stopping early gives fresh, fast tyres for longer, but those tyres must then survive to the end. Stopping twice gives fresher tyres throughout, but costs a second 20 to 25 seconds. The best plan is a balance between these effects, and that balance is what this project tries to find.

1.2 Problem statement

> **PROBLEM STATEMENT**
> Given real lap-time data from past races, build a model that predicts how a driver's lap time will change over the rest of a race for any tyre plan. Use that model to compute the total race time of every allowed pit strategy, recommend the fastest one, and state how certain that recommendation is.

The prediction is made early in the race, once the driver has completed a few clean laps. Nothing that happens later in the race is used. This matters, because a model that uses information from the future would look accurate in testing and be useless in practice.

1.3 The problem written as mathematics

Let a strategy be called s. It is a list of pit stops, each with a lap number and a tyre compound. Let l_0 be the first lap we simulate and L the last lap of the race. If $t_l(s)$ is the time of lap l under strategy s, $k(s)$ is the number of pit stops and P is the time lost per stop, the race time is

$$R(s) \;=\; \sum_{l=l_0}^{L} t_l(s) \;+\; Pk(s) \tag{1}$$

We do not know the true lap times $t_l(s)$ before they are driven. A model gives predictions $\hat{t}_l(s)$ (the hat means "predicted"), and so a predicted race time $\hat{R}(s)$. The recommended strategy is the one that makes the predicted race time smallest:

$$\hat{s} \;=\; \arg\min_{s \,\in\, \mathcal{S}} \hat{R}(s) \tag{2}$$

Here $\mathcal{S}$ is the set of all strategies the rules allow, and "arg min" means "the value of s at which the expression is smallest". Equation (2) contains the whole project in three parts:

Part	Question	Mathematical tool
Prediction	What will each lap time be?	Regression (Sections 3.2 to 3.9)
Optimisation	Which strategy has the smallest race time?	Search over a finite set (Section 3.11)
Uncertainty	How sure are we that it really is the fastest?	Variance and probability (Sections 3.12 to 3.14)

Table 1. The three parts of the problem.

1.4 What kind of problem this is

- **It is supervised regression.** We have examples where both the inputs (tyre age, compound, and so on) and the answer (the lap time) are known. The model learns the relation and predicts a number. Predicting a number is called regression; predicting a category would be classification.
- **It is not a time-series model.** A time-series model predicts the next value from the previous values of the same series. This model never uses the previous lap's time. It only uses quantities that are known in advance for any plan, which is what allows it to simulate strategies that were never driven.
- **It is not a physics model.** There are no equations for drag, downforce or engine power. The behaviour of the car is learned from measured lap times.

1.5 Objectives

- Build a clean data set of real racing laps from four seasons.
- Define a target that a model can learn across different circuits and cars.
- Compare simple reference models, linear regression and gradient boosting, and choose one using a season the models were not trained on.
- Test the chosen model once on a later, untouched season.
- Use the model to simulate and rank pit strategies.
- Measure the uncertainty of the predictions and say honestly what the simulator can and cannot decide.

SECTION 2

Data description

2.1 Source

The data is the official Formula 1 timing data for the 2022, 2023, 2024 and 2025 seasons. It was downloaded with FastF1, an open Python library that reads the timing feed published by Formula 1. It is real, measured data. Nothing in it is simulated.

These four seasons were chosen because they share one set of technical rules, so the cars are comparable from year to year. Each row of the raw data is **one lap driven by one driver in one race**. The columns used are listed in Table 2.

Column	Meaning	Example
Year, Round, Event	Which race	2022, round 1, Bahrain Grand Prix
Driver, Team	Who drove the lap	ALB, Williams
LapNumber	Lap of the race	10
LapTimeS	Lap time in seconds	103.500
Compound	Tyre fitted: soft, medium, hard, or a wet-weather tyre	SOFT
TyreLife	Laps completed on that set of tyres	10
Stint	Counts the sets of tyres used so far (1, 2, 3, ...)	1
TrackStatus	Green flag, yellow flag, safety car, and so on	1 (green)
PitIn, PitOut	Whether the lap entered or left the pit lane	False, False
TrackTemp, Rainfall	Weather at the time of the lap	28.4 °C, no rain

Table 2. Columns of the raw lap data that the project uses.

2.2 Cleaning: from raw laps to usable laps

Not every lap shows the true pace of the car. A lap behind the safety car is slow for reasons that have nothing to do with tyres. Such laps would teach the model the wrong thing, so they are removed. A lap is kept only if all of the following hold:

- the track was under a green flag for the whole lap;
- the car was on dry tyres (soft, medium or hard) and it was not raining;
- the lap did not enter or leave the pit lane;
- it is not the first lap of the race, which includes the standing start;
- the timing system marks the lap time as accurate;

- the lap is at most 4% slower than that driver's median clean lap in that race. Slower laps are treated as traffic, mistakes or left-over caution laps.

Whole races are also removed if more than 5% of their laps were driven on wet-weather tyres, because wet races follow different rules. Of 92 races downloaded, 12 were dropped as wet and 5 more had no usable laps after cleaning. This leaves **75 dry races** and **55,215 clean laps**, from 31 drivers at 25 circuits.

2.3 The baseline and the target

The most important design decision in the project is what the model is asked to predict. The obvious choice is the lap time in seconds, but that is a poor target. A lap takes about 75 seconds at one circuit and 105 seconds at another, and a fast car is seconds quicker than a slow one. A model trained on raw lap times would spend almost all of its effort learning which circuit and which car it is looking at, and that is not what a strategy needs.

A strategy needs to know how lap time **changes** during the race. So each driver's own pace early in the race is used as a reference, called the **baseline**, and every later lap is expressed as a percentage change from it.

Baseline

The baseline B is the median of the driver's first clean laps: up to 8 laps taken from laps 2 to 12 of the race, on the first set of tyres, with at least 3 laps required. The median is the middle value when the lap times are sorted. It is used instead of the mean because one unusually slow lap moves the mean but barely moves the median.

Target

For lap i with lap time t_i, the target is

$$y_i = 100\left(\frac{t_i}{B} - 1\right) \tag{3}$$

The unit of y_i is percent of baseline pace. A value of 0 means the lap was exactly as fast as the baseline. A negative value means the lap was **faster** than the baseline, and a positive value means it was slower.

> **WORKED EXAMPLE**
>
> In the 2022 Bahrain Grand Prix, driver ALB had a baseline of $B = 101.6345$ s. On lap 10 the lap time was $t = 103.500$ s.
>
> $$y = 100\left(\frac{103.500}{101.6345} - 1\right) = 100\,(1.01835 - 1) = 1.835\ \%$$
>
> That lap was 1.84% slower than the baseline. To go back from a percentage to a lap time, rearrange equation (3): $t = B\,(1 + y/100)$.

Dividing by the baseline removes the circuit and the car from the problem, because both are already inside B. What remains is the part that depends on tyres and fuel, and that part behaves similarly at

every circuit, so one model can learn from all races together.

2.4 Inputs to the model

The model has nine inputs, also called features. Six are numbers and three are categories.

Input	Type	Meaning
`tyre_life`	Number	Laps completed on the current set of tyres
`compound`	Category	Soft, medium or hard
`lap_frac`	Number	Share of the race completed (lap number divided by race length). It stands for the fuel load.
`laps_since_baseline_frac`	Number	Share of the race driven since the baseline laps, that is, fuel burned since the baseline
`total_laps`	Number	Race length in laps
`circuit`	Category	Which track
`baseline_compound`	Category	Tyre the baseline was set on
`baseline_tyre_life`	Number	Age of the tyres when the baseline was set
`baseline_track_temp`	Number	Track temperature when the baseline was set

Table 3. The nine inputs of the lap-time model.

The last three inputs describe the conditions under which the baseline was measured. They are needed because the baseline is not a neutral reference. A baseline set on new soft tyres is a fast reference, so later laps look slower by comparison than they would against a baseline set on old hard tyres.

> **WHY THESE INPUTS AND NOT OTHERS**
> Every input can be written down in advance for any plan, even a plan nobody has driven. If we decide to stop on lap 30 and fit hard tyres, we know the tyre age and compound on lap 40 without driving it. Inputs such as the gap to the car ahead would help the model on real laps, but they are unknown for a hypothetical strategy, so the simulator could not use them.

2.5 Training, validation and test sets

The data is divided into three parts by season. Each part has a different job.

Set	Seasons	Races	Driver-races	Laps	Job
Train	2022, 2023	35	623	24,039	The models learn from these laps
Validation	2024	19	356	15,332	Used to compare models and choose one
Test	2025	21	358	15,844	Used once, at the end, to report the final accuracy

Table 4. The split by season.

A “driver-race” is one driver in one race. It is the natural unit of this data, because all of a driver's laps in a race share the same baseline.

> **WHY THE SPLIT IS BY SEASON AND NOT RANDOM**
>
> A common approach is to shuffle all rows and take a random 80% for training and 20% for testing. That would be wrong here. Laps from the same driver in the same race are alike: if the model is too slow on one of them, it tends to be too slow on all of them (Section 3.12 measures this). A random split would place some laps of a race in the training set and the others in the test set, so the model would be tested on races it has partly seen, and its accuracy would look better than it is.
>
> Splitting by season guarantees that no race appears on both sides. It also matches how the simulator would be used: trained on past seasons and applied to a new one.

2.6 Pit-stop time loss

The simulator needs P, the time one pit stop costs, for each circuit. It is measured from real stops. For every stop made under a green flag, the lap entering the pit lane (the in-lap) and the lap leaving it (the out-lap) are compared with two normal laps of the same driver:

$$P_{\text{stop}} = t_{\text{in}} + t_{\text{out}} - 2\,\tilde{t} \tag{4}$$

where $\tilde{t}$ is the driver's median clean lap time in that race. Stops outside the range 10 to 45 seconds are discarded as repairs or penalties. P for a circuit is the median over all its stops in 2022 to 2024. The 2025 season is not used, so the test season stays untouched. The values lie between 19.6 s and 30.2 s, with an overall median of 23.0 s.

2.7 A first look at the target

Set	Mean of y (%)	Standard deviation of y (%)	Median lap time (s)
Train	−1.233	1.203	89.2
Validation	−1.181	1.221	85.6
Test	−1.219	1.287	84.1

Table 5. Summary of the target in each set.

Two facts in this table are used later. First, the mean is negative: on average, laps after the baseline are a little more than 1% **faster** than the baseline. Fuel burn makes the car lighter, and this effect is

on average larger than tyre wear. Second, the standard deviation is about 1.2%, which is roughly one second of lap time. That one second is all the variation there is to explain, and it is the scale against which the model must be judged.

SECTION 3

Methods and mathematics

This is the main section of the report. Each method is introduced with the idea first, then the equation, then a small example with numbers. The sections follow the order in which the simulator works: predict a lap, measure the error, choose a model, add laps into a race, search for the best strategy, and measure the uncertainty.

3.1 Notation

Symbol	Meaning	Unit
i	Index of a lap in the data set; n is the number of laps	–
t_i	Measured lap time	s
B	Baseline pace of the driver in that race	s
y_i	Target: change in pace from the baseline, equation (3)	%
$\mathbf{x}_i$	The nine inputs for lap i	–
$\hat{y}_i,\ \hat{t}_i$	Model predictions of y_i and t_i	%, s
$e_i = y_i - \hat{y}_i$	Error (residual) of the prediction	%
$\bar{y}$	Mean (average) of the y_i	%
s	A strategy: a list of pit laps and compounds	–
$R(s),\ \hat{R}(s)$	True and predicted race time of strategy s	s
$P,\ k(s)$	Pit-stop time loss; number of stops in s	s, –
N	Number of laps simulated	–
σ	Standard deviation; σ^2 is the variance	–

Table 6. Symbols used in this report.

3.2 Supervised regression

We assume that the target is produced by some unknown rule f applied to the inputs, plus a part that the inputs cannot explain:

$$y_i \;=\; f(\mathbf{x}_i) \;+\; \varepsilon_i \tag{5}$$

The term ε_i is called noise. In this project it contains everything that affects a lap time but is not among the nine inputs: traffic, a small driving mistake, a gust of wind, a driver deliberately saving tyres. Noise is not a flaw of the model. It is a property of the data, and no model can predict it.

Learning means using the training laps, where both $\mathbf{x}_i$ and y_i are known, to build a function $\hat{f}$ that is close to f. **Predicting** means computing $\hat{y} = \hat{f}(\mathbf{x})$ for a new lap. The whole question is how to

choose $\hat{f}$, and to answer it we first need a way to measure how good a prediction is.

3.3 Measuring error

The error of one prediction is the difference between the true value and the predicted value, $e_i = y_i - \hat{y}_i$. To describe the error over many laps with one number, three measures are used.

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{6}$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \qquad \mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\, y_i - \hat{y}_i| \tag{7}$$

- **MSE**, the mean squared error, squares each error and takes the average. Squaring makes every term positive, so errors in opposite directions cannot cancel, and it penalises large errors much more than small ones.
- **RMSE**, the root mean squared error, is the square root of the MSE. It has the same unit as y, so it can be read as a typical size of error.
- **MAE**, the mean absolute error, is the average size of the error ignoring its sign. It is less sensitive to a few large errors than the RMSE.

WORKED EXAMPLE

Take three laps with true values $y = -1.0,\ -2.0,\ 0.5$ and predictions $\hat{y} = -1.4,\ -1.5,\ 0.0$. The errors are 0.4, −0.5, 0.5.

$$\mathrm{MSE} = \frac{0.4^2 + 0.5^2 + 0.5^2}{3} = \frac{0.66}{3} = 0.22, \qquad \mathrm{RMSE} = \sqrt{0.22} = 0.469$$

$$\mathrm{MAE} = \frac{0.4 + 0.5 + 0.5}{3} = 0.467$$

From percent to seconds

An error in percent of pace is converted to seconds by multiplying by the baseline, because y was defined as a percentage of B:

$$e_i^{(\mathrm{s})} = \frac{e_i}{100}\,B \tag{8}$$

With a baseline of 90 seconds, an error of 1% is 0.9 seconds. Results in this report are given in both units.

3.4 Reference models

An error of 0.8 seconds per lap means nothing on its own. It needs something to be compared with. Two very simple models, which learn almost nothing, provide that comparison. Any useful model must beat them.

- **No change from baseline:** predict $\hat{y}_i = 0$ for every lap. This says the driver will keep lapping at the baseline pace for the rest of the race. It is what one would assume without any model.
- **Mean change:** predict $\hat{y}_i = \bar{y}_{\text{train}}$, the average of the target over the training laps (−1.233%), for every lap. This knows that cars get faster on average but nothing else.

3.5 Linear regression

Linear regression assumes that the target is a weighted sum of the inputs:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} \tag{9}$$

Each β_j is a number called a coefficient. β_0 is the intercept, the prediction when all inputs are zero. β_1 says how much the prediction changes when input 1 increases by one unit and the others stay fixed. If input 1 is tyre age, β_1 is the tyre wear per lap.

Choosing the coefficients: least squares

The coefficients are chosen to make the squared error on the training laps as small as possible:

$$\hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \tag{10}$$

This minimisation has an exact solution. Collect the inputs in a matrix X with one row per lap and one column per input (plus a column of ones for the intercept), and the targets in a vector $\mathbf{y}$. Setting the derivative of the sum with respect to each coefficient to zero gives a system of linear equations, called the normal equations, whose solution is

$$\hat{\beta} = (X^T X)^{-1} X^T \mathbf{y} \tag{11}$$

Two preparations of the inputs

Standardising. Each numeric input is rescaled to have mean 0 and standard deviation 1, by $z = (x - \bar{x})/s_x$. This does not change the predictions, but it puts the inputs on a common scale so the coefficients can be compared.

Dummy variables. A category such as the compound is not a number, so it cannot be multiplied by a coefficient. It is replaced by one 0/1 column per category: a column that is 1 when the tyre is soft and 0 otherwise, another for medium, another for hard. The coefficient of each column is then the shift in pace for that compound.

Two linear models were fitted:

- **Linear regression:** the six numeric inputs and dummy variables for the compound and the baseline compound. It has 13 fitted numbers in total: 12 slopes and one intercept.
- **Linear, wear per compound:** the same, but tyre age is also squared and multiplied by the compound dummies. This lets each compound have its own wear rate and its own curvature. It is fitted with a small ridge penalty, which adds $\alpha \sum_j \beta_j^2$ (with $\alpha = 1$) to the sum being minimised and so discourages very large coefficients.

LIMITATION OF A LINEAR MODEL

A linear model can only draw straight lines (or the specific curves we build by hand, such as the square of tyre age). It cannot discover by itself that tyre wear is steep for the first laps and then flattens, or that soft tyres behave differently at one circuit than at another. The next method can.

3.6 Regression trees

A regression tree predicts by asking a sequence of yes/no questions about the inputs, such as "is the tyre age at most 8 laps?". Each answer leads to another question or to a final group, called a **leaf**. Every lap ends in exactly one leaf, and the prediction for a leaf is the average target of the training laps that ended there.

Mathematically, a tree with J leaves divides the space of inputs into J regions $R_1, \dots, R_J$ and predicts a constant w_j in each:

$$h(\mathbf{x}) \;=\; \sum_{j=1}^{J} w_j \, \mathbf{1}[\mathbf{x} \in R_j] \tag{12}$$

The symbol $\mathbf{1}[\,\cdot\,]$ is 1 when the statement inside is true and 0 otherwise, so the sum simply picks the value w_j of the region that $\mathbf{x}$ falls in.

How a split is chosen

At each step the tree tries every input and every possible threshold, and keeps the split that reduces the squared error the most. The squared error of a group is the sum of squared distances from the group's mean:

$$\mathrm{SSE}(R) \;=\; \sum_{i \in R} (y_i - \bar{y}_R)^2, \qquad \text{gain} \;=\; \mathrm{SSE}(R) - \mathrm{SSE}(R_{\text{left}}) - \mathrm{SSE}(R_{\text{right}}) \tag{13}$$

WORKED EXAMPLE

Six laps have tyre ages 2, 5, 8, 20, 25, 30 and targets
$y = -1.6,\ -1.5,\ -1.3,\ -0.6,\ -0.5,\ -0.4$.

Before any split, the mean is −0.983 and the squared error around it is $\mathrm{SSE} = 1.468$.

Split at "tyre age at most 8". The left group (−1.6, −1.5, −1.3) has mean −1.467 and SSE 0.047. The right group (−0.6, −0.5, −0.4) has mean −0.500 and SSE 0.020.

$$\text{gain} \;=\; 1.468 - 0.047 - 0.020 \;=\; 1.401$$

One question removed 95% of the squared error. The tree now predicts −1.467 for young tyres and −0.500 for old tyres.

A tree handles categories directly. For the compound it can ask "is the tyre soft or medium?", with no dummy variables. It also captures interactions automatically: a question about tyre age asked **after** a question about the circuit gives a different wear rate for that circuit. The weakness of a single tree is that its prediction is a staircase with only J different values. A small tree is too crude, and a large

tree memorises the training data. Gradient boosting solves this by adding many small trees.

3.7 Gradient boosting

Gradient boosting builds the prediction in many small steps. It starts with a constant, looks at the errors that remain, fits a small tree to those errors, adds a fraction of that tree to the model, and repeats. Each tree corrects a little of what the previous trees got wrong.

The algorithm

Step 0. Start with the mean of the training targets:

$$F_0(\mathbf{x}) \;=\; \bar{y} \tag{14}$$

Step m, for $m = 1, 2, \ldots, M$. First compute what the current model still gets wrong on every training lap. These are the residuals:

$$r_i^{(m)} \;=\; y_i - F_{m-1}(\mathbf{x}_i) \tag{15}$$

Then fit a small tree h_m to the residuals, treating them as the target, and add a fraction ν of it to the model:

$$F_m(\mathbf{x}) \;=\; F_{m-1}(\mathbf{x}) \;+\; \nu\, h_m(\mathbf{x}) \tag{16}$$

After M steps the final model is the starting constant plus the sum of all the trees:

$$\hat{y} \;=\; F_M(\mathbf{x}) \;=\; \bar{y} \;+\; \nu \sum_{m=1}^{M} h_m(\mathbf{x}) \tag{17}$$

Why it is called “gradient” boosting

The method minimises a loss function. For regression the loss of one lap is half the squared error, $L(y, F) = \tfrac{1}{2}(y - F)^2$. Its derivative with respect to the prediction F is

$$\frac{\partial L}{\partial F} \;=\; -\,(y - F) \qquad \Longrightarrow \qquad -\frac{\partial L}{\partial F} \;=\; y - F \;=\; r \tag{18}$$

So the residual is exactly the negative gradient of the loss. The gradient points in the direction in which the loss increases fastest, and the negative gradient points downhill. Fitting a tree to the residuals and adding a small multiple of it is therefore a step of **gradient descent**, the same idea as walking downhill in small steps to find the bottom of a valley. The difference is that each step is a function (a tree) and not a number.

The value of a leaf

With squared-error loss, the best value for a leaf would be the mean of the residuals in it. The model used here adds a penalty λ that shrinks the value slightly towards zero:

$$w_j = \frac{\sum_{i \in R_j} r_i}{n_j + \lambda} \tag{19}$$

where n_j is the number of training laps in leaf j. With $\lambda = 0$ this is the plain mean. With $\lambda = 5$ and at least 300 laps per leaf, the value is reduced by at most $5/305 = 1.6\%$. The penalty matters most for small leaves, whose mean is the least reliable.

WORKED EXAMPLE: ONE BOOSTING STEP

Use the six laps of the previous example with $\nu = 0.03$ and, to keep the arithmetic simple, $\lambda = 0$.

Step 0: $F_0 = \bar{y} = -0.983$ for every lap.

Residuals: −0.617, −0.517, −0.317, 0.383, 0.483, 0.583. The first three laps are faster than the current prediction and the last three are slower.

The tree splits at tyre age 8. The leaf values are the mean residuals: −0.483 on the left and +0.483 on the right.

$$\text{young tyres}: \quad F_1 = -0.983 + 0.03 \times (-0.483) = -0.998$$

$$\text{old tyres}: \quad F_1 = -0.983 + 0.03 \times 0.483 = -0.969$$

The prediction has moved only 3% of the way towards the leaf means. The next tree is fitted to the new, slightly smaller residuals, and so on for 300 steps.

Why small steps

Taking only a fraction ν of each tree looks wasteful, but it is what makes the method work well. A single large step would fit the noise in the training data along with the real pattern. Many small steps let the model approach the pattern gradually, and averaging over many trees smooths out the accidents of each one.

Settings used

Setting	Symbol	Value	What it controls
Number of trees	M	300	How many correction steps are taken
Learning rate	ν	0.03	The fraction of each tree that is added
Leaves per tree	J	4	How complicated each tree is. Four leaves means at most three questions.
Minimum laps per leaf	–	300	A leaf must contain at least 300 training laps, so no value rests on a handful of laps
L2 penalty	λ	5	Shrinks leaf values towards zero, equation (19)

Table 7. Settings of the gradient-boosting model.

The implementation is `HistGradientBoostingRegressor` from the scikit-learn library. Before growing trees it groups each numeric input into at most 255 bins, which makes training fast without

changing the method described above. Because the data set has more than 10,000 laps, the library also sets aside 10% of the training laps to stop training early if the error stops improving. That never happened: all 300 trees were used.

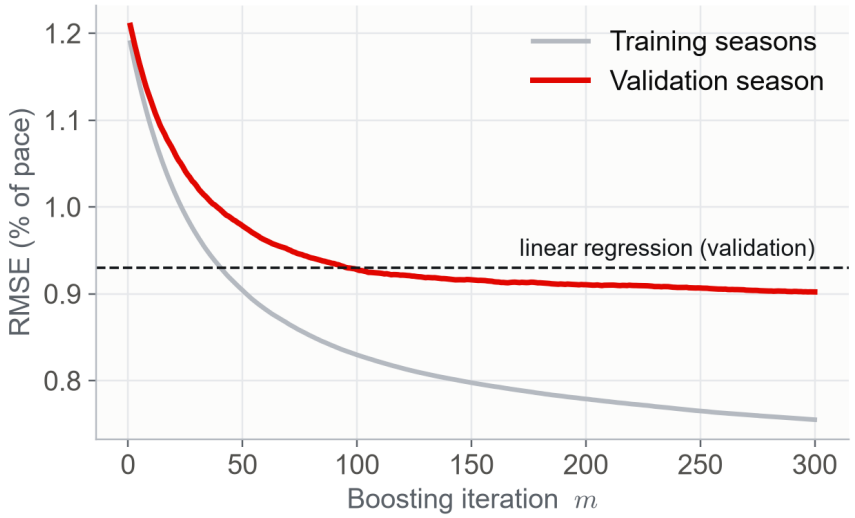


Figure 1. Error after each boosting step. The error on the training seasons keeps falling. The error on the validation season falls quickly, passes linear regression after about 100 trees, and then flattens.

3.8 Overfitting and model selection

Figure 1 shows the central difficulty of predictive modelling. The error on the training data always falls as the model becomes more flexible, because a flexible model can bend to fit every point. But part of what it fits is noise, which will be different in the next race. Fitting noise is called **overfitting**, and it makes predictions on new data worse.

The expected squared error of a prediction on a new lap can be split into three parts:

$$\mathrm{E}\big[(y-\hat{y})^2\big] \;=\; \mathrm{bias}^2 \;+\; \mathrm{variance} \;+\; \sigma_\varepsilon^2 \tag{20}$$

- **Bias** is the error from a model that is too simple to follow the real pattern. A straight line fitted to a curve has bias.
- **Variance** is the error from a model that is too sensitive to the particular training data it saw. Train it on different races and it would give a noticeably different answer.
- **Noise** σ_ε^2 is the part no model can remove.

A simple model has high bias and low variance. A flexible model has low bias and high variance. The best model is in between, and the training data cannot tell us where, because training error only ever goes down. This is the reason for the **validation set**: a season the models have not seen, on which their errors can be compared fairly.

Procedure followed

- Fit every candidate model on the training seasons (2022 and 2023).
- Compute each model's error on the validation season (2024).
- Choose the model with the smallest validation error.
- Fit the chosen model again on 2022 to 2024 together, so it learns from as much data as possible.

- Measure its error once on the test season (2025).

The test season is used only once, at the very end. If it were used to choose between models, the reported accuracy would be optimistic, because the choice itself would have been tuned to that season.

3.9 The coefficient of determination, R²

R^2 compares the model's squared error with the squared error of the simplest possible prediction, the mean:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{21}$$

- $R^2 = 1$: the model is perfect.
- $R^2 = 0$: the model is exactly as good as always predicting the mean.
- $R^2 < 0$: the model is **worse** than always predicting the mean. This is possible on data the model was not fitted to, and it is not a mistake in the calculation.

R^2 depends on what the model is compared with. In this project the same predictions give three different values, and all three are correct. The numerator, the model's own error, is identical in each. Only the denominator changes.

Version	Compared with	Test value	What it tells us
R² of lap time	The mean lap time over all circuits	0.992	Little. Lap times differ by about 10 s between circuits and the baseline already contains that. Constant pace scores 0.973.
R² of pace change	The mean of the target y	0.436	How much of the movement of pace within a race the model explains. This is the demanding version and the one used to choose the model.
Error removed	Constant baseline pace ($\hat{y} = 0$)	0.713	How much better the model is than using no model at all.

Table 8. Three versions of R² for the same predictions (2025 test season).

Why “no change” has a negative R²

For the prediction $\hat{y}_i = 0$ the mean squared error is the average of y_i^2 . The average of a square can always be split into the square of the mean plus the variance:

$$\frac{1}{n}\sum_i y_i^2 = \bar{y}^2 + s_y^2 \tag{22}$$

On the validation season, $\bar{y} = -1.181$ and $s_y = 1.220$:

$$\mathrm{MSE}_0 = 1.181^2 + 1.220^2 = 1.394 + 1.490 = 2.884 = 1.698^2$$

$$R_0^2 = 1 - \frac{\bar{y}^2 + s_y^2}{s_y^2} = -\frac{\bar{y}^2}{s_y^2} = -\frac{1.394}{1.490} = -0.936 \tag{23}$$

The "no change" model is wrong on average by 1.18%, because it ignores the fact that cars get faster as fuel burns. That constant mistake, the bias, is why it scores below zero. The "mean change" model scores −0.002 and not exactly 0 for a similar but much smaller reason: it predicts the mean of the training seasons (−1.233%), which differs slightly from the mean of the validation season (−1.181%).

3.10 From one lap to a whole race

The model predicts a percentage. Three steps turn it into a race time. First, the percentage becomes a lap time by inverting equation (3):

$$\hat{t}_l(s) = B\left(1 + \frac{\hat{y}_l(s)}{100}\right) \tag{24}$$

Second, the lap times of the remaining laps are added. Third, the pit-stop time loss is added once per stop:

$$\hat{R}(s) = \sum_{l=l_0}^{L} \hat{t}_l(s) + Pk(s) \tag{25}$$

The strategy s enters through the inputs. Choosing a pit lap and a compound fixes the tyre age and the compound on every later lap, and those are inputs of the model. For example, with a stop on lap 30 for hard tyres, lap 31 has compound hard and tyre age 1, lap 32 has tyre age 2, and so on. The simulation starts on the lap after the baseline window ends (l_0), which is at most lap 13.

3.11 Optimisation: searching the strategies

We now want the strategy with the smallest $\hat{R}(s)$. In many optimisation problems the unknown is a real number and calculus is used: set the derivative to zero. Here the unknowns are a lap number (a whole number) and a compound (a category), and the model is a staircase function with no useful derivative. But the number of possible strategies is finite and small, so the simplest method is also the best one: **compute the race time of every allowed strategy and take the smallest**. This is called exhaustive search, and it is guaranteed to find the true minimum of the predicted race time.

Which strategies are allowed

A strategy belongs to the allowed set $\mathcal{S}$ if it satisfies three conditions:

- **Every stint is at least 5 laps long.** A stint is the run of laps on one set of tyres. Shorter stints are never sensible.
- **At least two different dry compounds are used**, as the rules require.
- **No stint is longer than tyres have been seen to last.** The limit is the 98th percentile of tyre age observed on that compound at that circuit. A tree model cannot extrapolate: beyond the largest tyre age in its data it simply repeats its last value, which would make very old tyres look

harmless.

Counting the candidates

Take the example race used in Section 4: simulation from lap 12 to lap 50. With stints of at least 5 laps, the pit lap of a one-stop strategy can be any lap from 16 to 45, which is 30 laps. With 3 compounds to fit:

$$\text{one stop}: \quad 30 \times 3 \;=\; 90 \text{ candidates}$$

For two stops, the pit laps are searched on every second lap (16, 18, ..., 44), which gives 15 possible laps. The two stops must be at least 5 laps apart, so on this grid they must be at least 3 grid steps apart. The number of such pairs is $12 + 11 + \cdots + 1 = 78$, and each stop can fit any of 3 compounds:

$$\text{two stops}: \quad 78 \times 3 \times 3 \;=\; 702 \text{ candidates}$$

Of these 792 candidates, 497 satisfy all three conditions (23 one-stop and 474 two-stop). Each needs 39 lap predictions, so the search is about 19,000 predictions and takes well under a second.

3.12 The structure of the error

So far each lap's error has been treated separately. To say how uncertain a **race time** is, we need to know how the errors of different laps are related, because a race time is a sum of many laps.

If the model is too optimistic about a driver on one lap, it is usually too optimistic about that driver for the whole race. The errors are not independent. They are modelled as the sum of three parts, for lap l in stint j of driver-race d:

$$e_{djl} \;=\; a_d \;+\; b_{dj} \;+\; \varepsilon_{djl} \tag{26}$$

Part	Shared by	Typical cause	Standard deviation (% of pace)
a_d	All laps of one driver in one race	The baseline was a little too fast or too slow	$\sigma_a = 0.545$
b_{dj}	All laps of one stint	That set of tyres worked better or worse than usual	$\sigma_b = 0.438$
ε_{djl}	One lap only	Traffic, a small mistake, wind	$\sigma_\varepsilon = 0.650$

Table 9. The three parts of the prediction error, measured on the 2024 validation season.

How the three parts are measured

- σ_a : average the error over each driver-race, then take the standard deviation of those averages.
- σ_b : average the error over each stint, subtract the driver-race average, then take the standard deviation.
- σ_ε : subtract the stint average from each lap's error, then take the standard deviation.

Variances add

If the three parts are independent of each other, the variance of their sum is the sum of their variances:

$$\sigma_e^2 \;=\; \sigma_a^2 + \sigma_b^2 + \sigma_\varepsilon^2 \;=\; 0.297 + 0.192 + 0.423 \;=\; 0.912 \tag{27}$$

Note that it is the variances (squares) that add, not the standard deviations. As shares of the total: 33% of the error variance belongs to the whole race, 21% to the stint and 46% to the single lap. (The sum 0.912 is a little larger than the directly measured mean squared error 0.814, because the simple method above slightly overestimates the two group-level parts: an average over a group of laps still contains some lap noise.)

Intraclass correlation

The share of the variance that is common to a whole driver-race is called the intraclass correlation:

$$\rho \;=\; \frac{\sigma_a^2}{\sigma_a^2 + \sigma_b^2 + \sigma_\varepsilon^2} \;=\; \frac{0.297}{0.912} \;=\; 0.33 \tag{28}$$

It is the correlation between the errors of two laps of the same driver in the same race but in different stints. A value of 0 would mean the laps are independent. The measured value of 0.33 means they are clearly not.

Effective sample size

Correlated observations carry less information than independent ones. If each group has m laps and correlation ρ, the data set is worth only

$$n_{\text{eff}} \;=\; \frac{n}{1 + (m-1)\,\rho} \;=\; \frac{15{,}332}{1 + (43.1 - 1) \times 0.326} \;=\; \frac{15{,}332}{14.7} \;\approx\; 1{,}043 \tag{29}$$

independent laps. The validation season has 15,332 laps, but for judging the model it is worth only about 1,040. This is the mathematical reason behind two earlier decisions: the split by season instead of at random (Section 2.5), and the use of small trees, since there is much less independent information than the number of rows suggests.

3.13 Uncertainty of a race time

A race time is a sum of lap times, so its error is a sum of lap errors. Two rules for variances are needed.

> **TWO RULES FOR THE VARIANCE OF A SUM**
>
> **Rule 1: independent errors.** If N errors are independent and each has variance σ^2, their sum has variance $N\sigma^2$. The standard deviation of the sum is $\sqrt{N}\,\sigma$. Independent errors partly cancel.
>
> **Rule 2: one shared error.** If the **same** error b is added on each of N laps, the total is Nb, with variance $N^2\sigma^2$ and standard deviation $N\sigma$. A shared error does not cancel at all.
>
> Over 39 laps the difference is large: $\sqrt{39} = 6.2$ against 39. This is why the shared parts of the error dominate the uncertainty of a race time, even though the lap-level part is the largest for a single lap.

Apply the rules to equation (26). The race-level part a_d is shared by all N laps. The stint-level part of stint j is shared by its n_j laps. The lap-level part is independent on every lap. Converting from

percent to seconds with the factor $B/100$:

$$\sigma_R^2 \;=\; \left(\frac{B}{100}\right)^2 \left[\, N^2\sigma_a^2 \;+\; \sigma_b^2 \sum_j n_j^2 \;+\; N\sigma_\varepsilon^2 \,\right] \tag{30}$$

WORKED EXAMPLE

Example race: $B = 93.85$ s and $N = 39$ laps. The recommended strategy has two stints of 22 and 17 laps.

$$\text{race}: \quad N^2\sigma_a^2 = 39^2 \times 0.2967 = 451.3$$

$$\text{stints}: \quad \sigma_b^2 \sum_j n_j^2 = 0.1917 \times (22^2 + 17^2) = 148.2$$

$$\text{laps}: \quad N\sigma_\varepsilon^2 = 39 \times 0.4231 = 16.5$$

$$\sigma_R \;=\; \frac{93.85}{100}\sqrt{451.3 + 148.2 + 16.5} \;=\; 23.3 \text{ s}$$

The lap-level noise, the largest part for a single lap, contributes only 16.5 of 616.0. Almost all the uncertainty of the race time comes from the two shared parts.

3.14 Deciding between two strategies

Suppose the simulator says strategy A is faster than strategy B by $\Delta = \hat{R}(B) - \hat{R}(A)$ seconds. Is A really faster? That depends on how large Δ is compared with the uncertainty of the difference.

The race-level error cancels

Both strategies are for the same driver in the same race, so both contain the same race-level error a_d on the same number of laps. When one race time is subtracted from the other, that part disappears. A baseline that is slightly wrong shifts every strategy by the same amount and does not change which one is best. Only the stint-level and lap-level parts remain:

$$\sigma^2_{\hat{R}(s)} \;=\; \left(\frac{B}{100}\right)^2 \left[\, \sigma_b^2 \sum_j n_j^2 + N\sigma_\varepsilon^2 \,\right], \qquad \sigma^2_\Delta \;=\; \sigma^2_{\hat{R}(A)} + \sigma^2_{\hat{R}(B)} \tag{31}$$

The second formula treats the two strategies' remaining errors as independent, which is Rule 1 again.

From a difference to a probability

If the error of the difference follows a normal distribution (the bell-shaped curve) with standard deviation σ_Δ, the probability that A is truly faster is

$$\Pr(A \text{ faster}) \;=\; \Phi\!\left(\frac{\Delta}{\sigma_\Delta}\right) \tag{32}$$

$\Phi(z)$ is the standard normal distribution function: the probability that a standard normal variable is at most z. $\Phi(0) = 0.5$, $\Phi(1) = 0.84$ and $\Phi(2) = 0.98$. So a predicted gain equal to one standard deviation gives 84% confidence, and a gain that is small compared with σ_Δ gives a probability close to 50%, which is no better than tossing a coin.

Monte Carlo simulation

The same probability can be estimated without the formula, by simulating many random races. This is called the Monte Carlo method. For each simulated race:

- draw one random stint offset for each stint from a normal distribution with standard deviation σ_b;
- draw one random noise value for each lap with standard deviation σ_ε;
- add them to the predicted change in pace, convert to lap times with equation (24), and add up the race time with equation (25).

Repeating this M times gives M possible race times for each strategy. The share of repetitions in which A beats B is the estimate $\hat{p}$ of the probability. Because it is based on a finite number of random draws, the estimate has its own uncertainty, the standard error:

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}\,(1-\hat{p})}{M}} \tag{33}$$

Using both the formula and the simulation is a check: if they agree, the algebra in equations (30) and (31) is right. The app uses the simulation to draw the spread of each strategy.

SECTION 4

Results

4.1 Model comparison on the validation season

All models were fitted on 2022 and 2023 and compared on 2024.

Model	RMSE (% of pace)	RMSE (s per lap)	MAE (s per lap)	R^2 pace	R^2 lap time
Gradient boosting	**0.902**	**0.798**	**0.588**	**0.454**	**0.994**
Gradient boosting, tyre wear forced to increase	0.909	0.803	0.593	0.445	0.994
Linear regression	0.930	0.822	0.624	0.419	0.994
Linear, wear per compound	0.938	0.828	0.628	0.410	0.994
Mean change	1.222	1.084	0.867	−0.002	0.989
No change from baseline	1.698	1.550	1.262	−0.936	0.978

Table 10. Validation results (2024), best model first.

- **Gradient boosting has the smallest error** and was selected.
- The margin over linear regression is small: 0.798 s against 0.822 s per lap. Most of what can be learned from these inputs is close to linear.
- Forcing predicted lap time never to improve as tyres age made the fit slightly worse, and so did giving each compound its own curve in the linear model.
- Every real model is far better than the two references. Assuming constant pace has an error of 1.550 s per lap.
- R^2 of lap time is above 0.97 for every row, including the reference models. It cannot tell the models apart, which is why R^2 of pace is used.

4.2 How complex should the trees be?

The number of leaves per tree was varied with all other settings fixed.

Leaves per tree	2	3	4	6	8	12	16	31	63
Training RMSE (%)	0.843	0.784	0.755	0.724	0.706	0.684	0.670	0.639	0.623
Validation RMSE (%)	0.926	0.905	0.902	0.913	0.921	0.923	0.933	0.940	0.939

Table 11. Error against tree size. Linear regression has a validation RMSE of 0.930%.

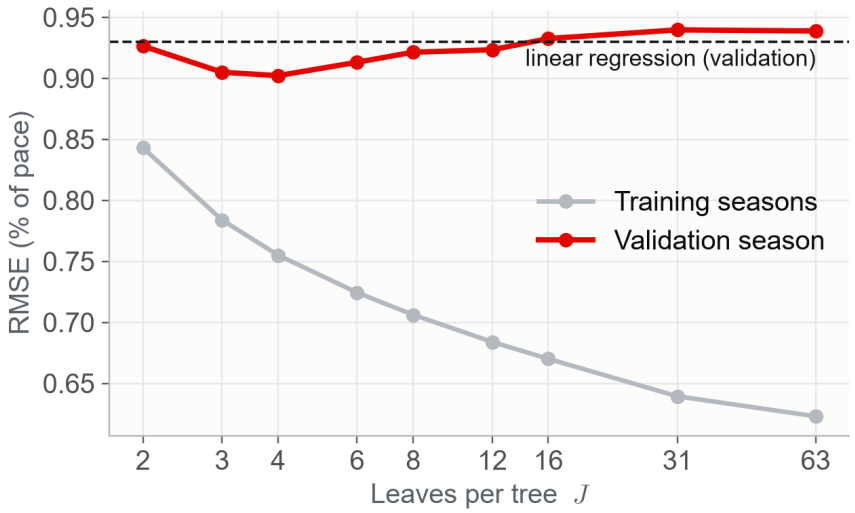


Figure 2. Training error keeps falling as trees grow, while validation error is lowest at 4 leaves and then rises above linear regression.

This is the bias–variance trade-off of Section 3.8 in real data. Trees with 2 leaves are too simple (bias). Trees with 16 or more leaves fit the training seasons much better but predict the new season **worse than linear regression** (variance). The library's default settings, with 31 leaves, give a validation RMSE of 0.929%. The reason is the one found in Section 3.12: the data contains far less independent information than its row count suggests, so a flexible model quickly starts fitting individual races.

Adding the team, the driver or the stint number as extra inputs was also tried. None of them reduced the validation error, so the nine inputs were kept.

4.3 Final test on the 2025 season

The selected model was fitted again on 2022 to 2024 and then applied once to 2025, a season it had never seen.

Model	RMSE (% of pace)	RMSE (s per lap)	MAE (s per lap)	R^2 pace	R^2 lap time
Gradient boosting	**0.966**	**0.835**	**0.625**	**0.436**	**0.992**
No change from baseline	1.773	1.558	1.249	−0.896	0.973

Table 12. Test results (2025).

- The typical lap-time error is **0.835 s** (RMSE), or 0.625 s on average (MAE), on laps of about 90 seconds.
- Assuming constant pace gives 1.558 s. The model removes **71%** of that squared error: $1 - (0.835/1.558)^2 = 0.713$.
- R^2 of pace is 0.436, close to the validation value of 0.454. The model performs on a new season about as well as it did on the season used to choose it, which is the sign of a model that has not been overfitted.

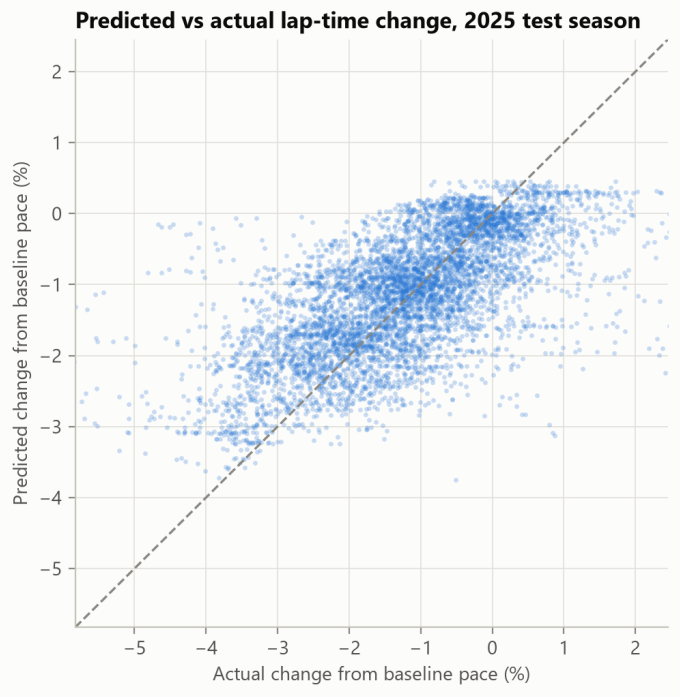


Figure 3. Predicted against actual change in pace for the 2025 laps. Points on the dashed line are perfect predictions. The cloud follows the line, but the predictions cover a narrower range than the actual values: the model predicts the expected lap and cannot predict the noise.

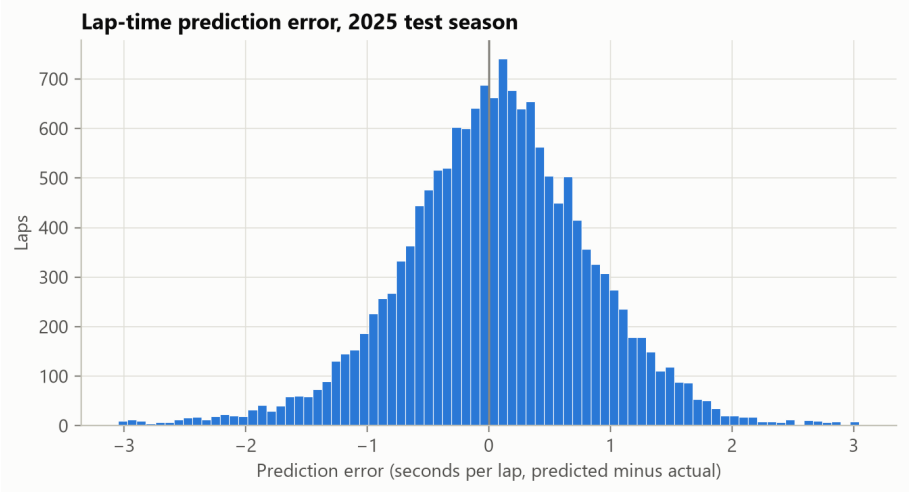


Figure 4. Distribution of the lap-time error on the 2025 season. It is centred close to zero and roughly symmetric, with most laps within one second.

4.4 What the model learned about tyres

Figure 5 shows the model's learned tyre wear. A set of tyres loses about 1% of pace, roughly one second per lap, over its first 20 laps. The soft tyre is the fastest at every age. Two features of the curves are limitations and not physics. The steps come from the tree structure: the prediction is a staircase. The flat part after about 20 laps is where the data runs thin, and it is too optimistic: very old tyres are only seen in the data when they happened to last (see Section 5.2).

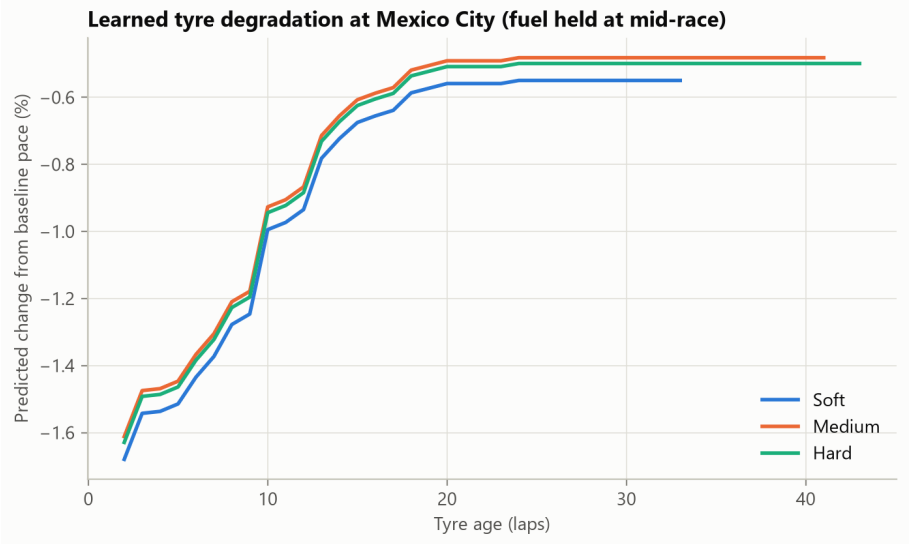


Figure 5. Predicted change in pace as a tyre ages, at one circuit, with fuel load held fixed. Lower is faster.

4.5 Structure of the error

Level	Standard deviation (%)	Variance	Share of variance
Whole driver-race, a_d	0.545	0.297	33%
Stint, b_{dj}	0.438	0.192	21%
Single lap, ε_{djl}	0.650	0.423	46%

Table 13. The three parts of the validation error.

Nearly half of the error (46%) is lap-to-lap noise, which no input known in advance could explain. This puts a ceiling on R^2 of pace well below 1 for any model using this kind of input. A third (33%) is a race-level offset, mostly from the baseline, with intraclass correlation $\rho = 0.33$ and an effective sample size of about 1,043 laps.

4.6 Race-time prediction

The simulator was run for every top-10 finisher with usable data in the dry 2025 races: 182 driver-races in 20 races. For each one it was given the strategy the team actually used, and its predicted race time was compared with the real one. This comparison is only fair for races with no safety car, since safety cars are not modelled; 60 driver-races qualify.

Quantity	Value
Mean absolute error of race time	24.3 s (0.57% of race time)
Root mean squared error of race time	27.9 s
Average error (bias)	+2.2 s
Mean absolute error assuming constant baseline pace	54.6 s
Standard deviation predicted by equation (30)	28.0 s

Table 14. Race-time prediction for the real strategy, green-flag races of 2025.

Over a race of about 70 minutes the simulator is wrong by 24 seconds on average, less than half the error of assuming constant pace.

The last row is a test of the theory in Section 3.13. Equation (30) was built from lap errors measured in 2024. It predicts that race-time errors in 2025 should have a spread of 28.0 s. The spread actually observed is 27.9 s. The two agree, so the three-part error model describes how lap errors add up into race errors.

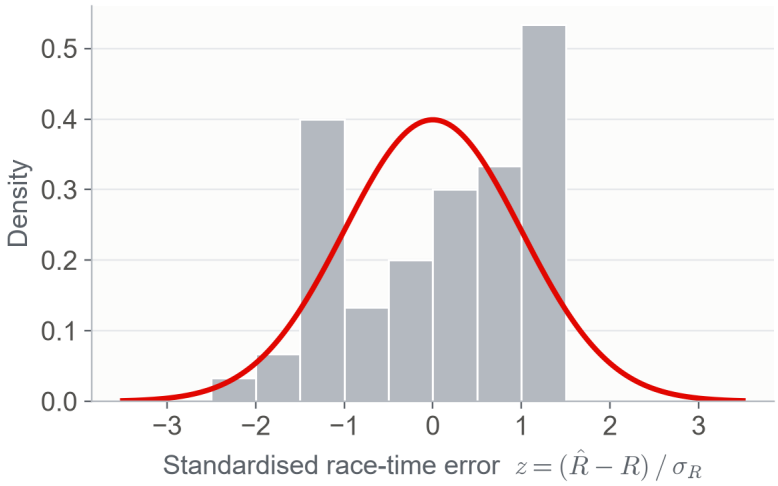


Figure 6. Race-time errors divided by their predicted standard deviation, with the standard normal curve. The spread matches (standard deviation 1.03, 98% within ±2), but with only 60 driver-races the shape is rough and is not shown to be normal.

4.7 Recommended strategy against the teams' choices

Quantity	Value
Same number of stops as the team	65%
Average distance between recommended and real first pit lap	8.0 laps
Recommended first pit lap within 3 laps of the team's	26%
Recommended stop is later than the team's	66% of cases, by 4.3 laps on average
Median predicted gain of the recommended strategy	2.2 s

Table 15. Recommended strategies compared with what teams did, 2025.

The simulator usually agrees with the teams on the number of stops, but not on the lap. It tends to recommend stopping later, and the gain it predicts from its own recommendation is small, about 2 seconds. Section 4.8 shows on one race why such a gain cannot be trusted.

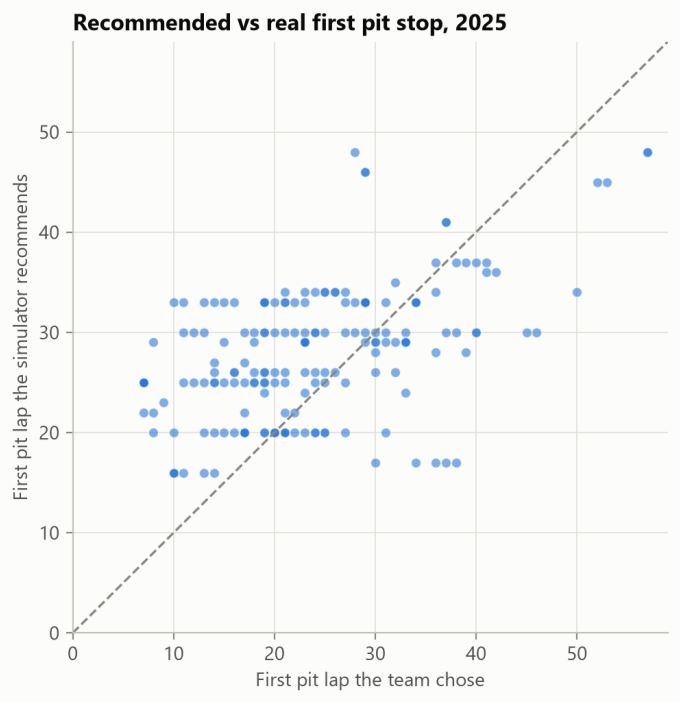


Figure 7. Recommended against real first pit lap for the 2025 driver-races. Points above the dashed line are cases where the simulator recommends a later stop than the team made.

4.8 A worked race: 2025 Saudi Arabian Grand Prix

Driver PIA, who won the race. The simulation starts on lap 12 of 50, on medium tyres, with baseline $B = 93.85$ s and pit loss $P = 21.91$ s.

	Strategy	First stint	Second stint
Team	Medium, stop on lap 19, then hard	8 laps	31 laps
Simulator	Medium, stop on lap 33, then soft	22 laps	17 laps

Table 16. The two strategies compared. Stint lengths are counted from the first simulated lap.

The simulator predicts that its strategy is faster by $\Delta = 3.2$ s. Now apply Section 3.14. The standard deviation of each strategy's race time, without the race-level part, comes from equation (31):

$$\sigma_{\text{sim}} = \frac{93.85}{100}\sqrt{0.1917\,(22^2 + 17^2) + 39 \times 0.4231} = 12.0 \text{ s}$$

$$\sigma_{\text{team}} = \frac{93.85}{100}\sqrt{0.1917\,(8^2 + 31^2) + 39 \times 0.4231} = 13.7 \text{ s}$$

$$\sigma_{\Delta} = \sqrt{12.0^2 + 13.7^2} = 18.2 \text{ s}$$

$$\Pr(\text{simulator faster}) \;=\; \Phi\!\left(\frac{3.2}{18.2}\right) \;=\; \Phi(0.18) \;=\; 0.57$$

A Monte Carlo simulation with $M = 4000$ races gives 0.578, with standard error

$$\mathrm{SE}(\hat{p}) \;=\; \sqrt{\frac{0.578 \times 0.422}{4000}} \;=\; 0.008$$

The formula and the simulation agree.

WHAT THIS RESULT MEANS

The simulator's strategy is predicted to be 3.2 s faster, but the uncertainty of that difference is 18 s. The probability that it really is faster is 57%, only a little better than a coin toss. The model cannot separate these two strategies.

This is not special to this race. 15 different strategies are within 2 seconds of the best one here, and 23 within 5 seconds. The predicted race time is almost flat near its minimum.

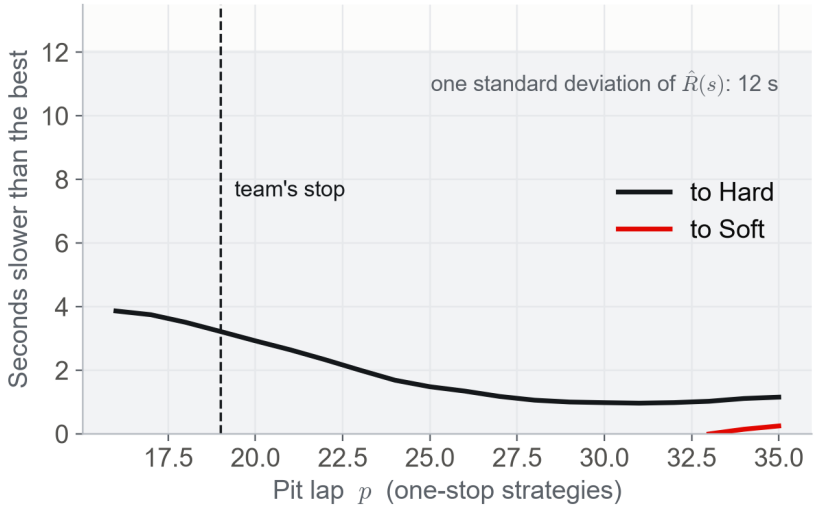


Figure 8. Predicted time lost against the best strategy, for one-stop strategies in the example race, as the pit lap changes. The whole curve lies inside one standard deviation of the race-time prediction (shaded).

SECTION 5

Conclusion

5.1 What was found

The question was whether a model, a few laps into a race, can predict the rest of it well enough to choose the pit strategy. The answer has two parts.

> ANSWER
>
> **Predicting the race: yes.** On a season the model never saw, the lap-time error is 0.835 s, which removes 71% of the error of assuming constant pace. Total race time is predicted within 0.57% (24 s), less than half the error of the constant-pace assumption.
>
> **Choosing the exact pit lap: no.** Good strategies differ by a few seconds, and the uncertainty of the difference between two strategies is about 18 seconds. The simulator can rule out clearly bad strategies and usually agrees with teams on the number of stops, but it cannot say which of several good strategies is best.

The main points behind this answer are:

- **The choice of target mattered more than the choice of model.** Predicting the percentage change from the driver's own baseline removed the circuit and the car from the problem and made one model usable for every race.
- **A small model was the best model.** Gradient boosting with trees of only 4 leaves beat linear regression by a small margin. Larger trees did worse than linear regression on new data.
- **The data is worth less than its size.** Errors within a driver-race are correlated ($\rho = 0.33$), so 15,332 validation laps carry the information of about 1,043 independent ones. This explains why small models win and why the split must be by season.
- **R^2 must be read with its reference.** The same predictions give 0.99 on lap time and 0.44 on the change in pace. The second is the honest measure of what the model adds.
- **The uncertainty can be calculated.** A three-part error model, with the rules for the variance of a sum, predicted a race-time spread of 28.0 s for 2025. The observed value was 27.9 s.
- **Stating the uncertainty changed the conclusion.** Without it, the simulator appears to beat the teams by a couple of seconds in most races. With it, that apparent gain is seen to be well inside the noise.

5.2 Limitations

- **No traffic or track position.** Teams often stop early to get ahead of a rival or to rejoin on a clear track. The simulator finds the fastest strategy for a car alone on the circuit.
- **No safety cars or changes in weather.** These decide many real races and are not modelled.
- **Old tyres look better than they are.** A long stint appears in the data only when the tyres lasted. Stints that were going badly were ended early and never reached a high tyre age. The model therefore learns from the survivors, is too optimistic about old tyres, and recommends later

stops than teams make (66% of cases).

- **Compound names are not constant.** “Soft” is a different rubber at different circuits. The model sees only the name and the circuit.
- **The team's choice is not the true optimum.** It is a reference from experts with more information, so disagreeing with it is not automatically an error.
- **The normal distribution is assumed, not shown.** The spread of race-time errors matches the theory, but 60 races are too few to confirm the bell shape.

5.3 Possible extensions

- **Update the baseline during the race.** A third of the error is a race-level offset. Re-estimating the baseline as more laps are observed should reduce it. This changes the question from predicting the whole race in advance to updating the forecast lap by lap.
- **Correct for surviving tyres.** Methods from survival analysis treat a stint that was ended early as incomplete information and not as missing, which would address the optimism about old tyres.
- **Model the other cars.** Adding traffic and the pit-stop choices of rivals would turn the problem from optimisation into a game between teams.
- **Model safety cars as random events,** so that a strategy is judged by its expected race time over many possible races.

5.4 Reproducing the results

All results come from the scripts in the project folder, run in this order:

Script	What it does
`01_download_data.py`	Downloads the race laps with FastF1
`02_build_dataset.py`	Cleans the laps, computes baselines, the target and pit loss
`03_train_models.py`	Compares the models, runs the final test, saves the model
`04_backtest.py`	Runs the simulator on the 2025 races
`05_app.py`	The interactive simulator (run with Streamlit)
`report/build_report.py`	Builds this report from the outputs of the scripts above